

CSE306 project 1 report

Cristian Verde

1 Overview

The objective of this project was to write a basic image renderer using ray tracing. The main C++ objects are a Scene and an Object which is extended by TriangleMesh and Sphere. The Scene is equipped with a camera position and a light position, together with a light intensity. Additionally, we have a lens diameter and a focal distance for depth of field, and a camera shutter time for motion blur (which are all optional but provide nice images).

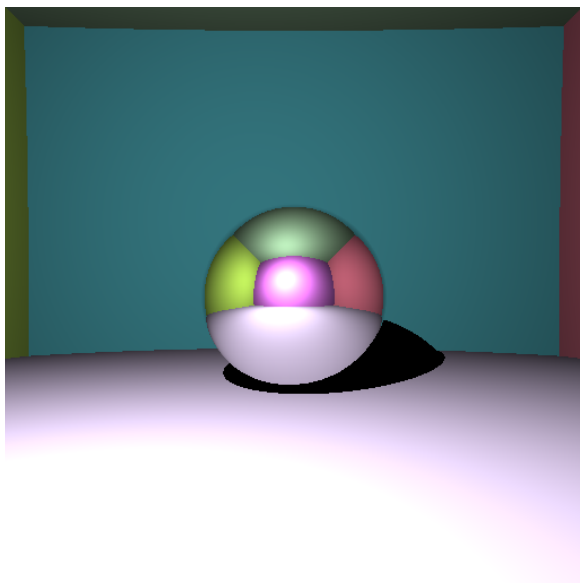
Additionally, each Object can have a velocity vector (for motion blur), and a `is_light` boolean variable to create non-point light sources (a spherical light source, for instance).

2 Results

The following images (and gif) were generated on my local machine running an Intel i7-13700H with 14 cores (6 fast with 2 threads each and 8 with only 1 thread, totalling 20 threads). The times are collected using the standard `time` program and each one is ran only once because I did not observe significant deviations. The compilation flags relevant for performance were `-O3 -fopenmp -ffast-math`.

For ease, I will time everything with 64 samples (apart from lab 1), a 512x512 picture and 5 light bounces (if not specified otherwise).

2.1 Lab 1



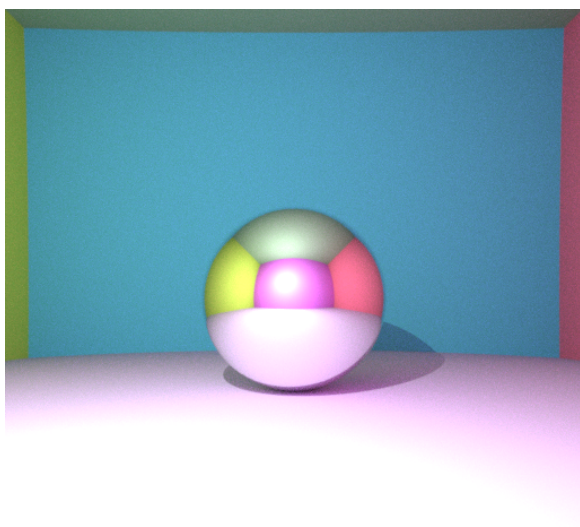
Execution metrics:

```
real 0m0.027s  
user 0m0.132s  
sys 0m0.003s
```

Figure 1: Result at the end of lab 1.

There is not much to say about this one. We send a ray in each direction (which is actually the ray that enters the camera, but “tracing the steps back” is what helps us see where the ray came from). We see what object (if any) is hit first, and if there is another object between this object and the light source, we have a shadow. The edges are ragged because there is no sampling yet, so each pixel “belongs” to at most one object (there is no blending). The generation time is also really fast because there is no sampling.

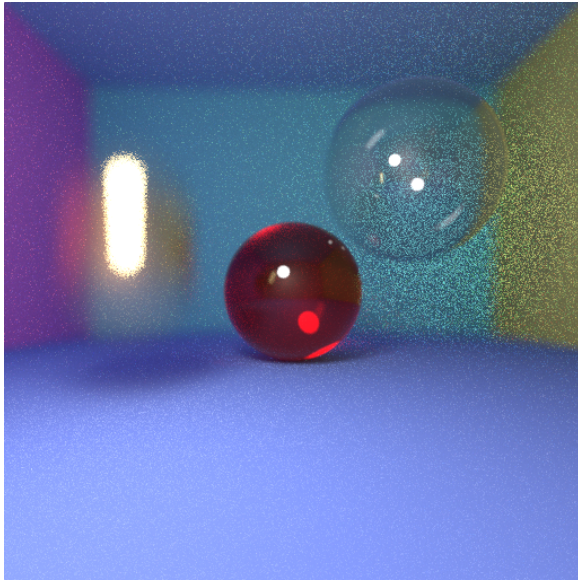
2.2 Lab 2



Execution metrics:

```
real 0m3.574s  
user 1m9.224s  
sys 0m0.030s
```

Figure 2: Result at the end of lab 2.

**Execution metrics:**

```
real 2m44.396s
user 54m13.604s
sys 0m1.163s
```

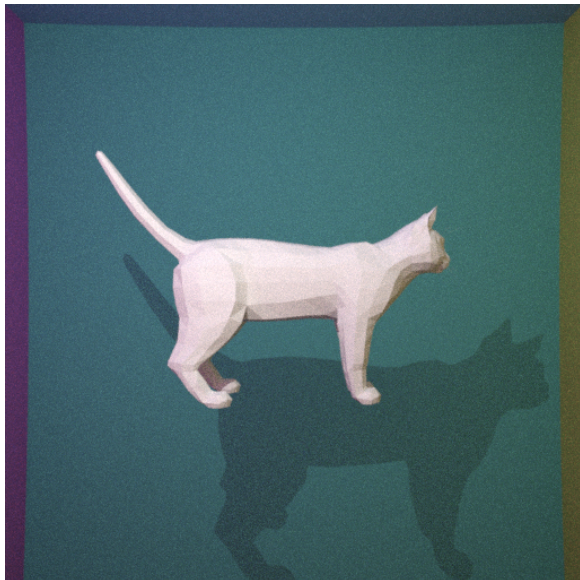
Figure 3: Lab 2 made cooler (extra work).
Also, 500 samples and 10 light bounces.

In this lab, we added sampling for anti-aliasing. This means we now shoot 64 rays in each direction, but with a slight deviation, and we average all the colors we get, to even out the edges (this basically blends the colors at the edges). We also add an indirect light component for “softer” shadows (otherwise they would be pitch black) by shooting a ray from the collision point.

The result is about 500 times slower than before, due to 64×5 more operations, and some cache misses due to our non-`thread_local` random number generation.

Disclaimer, everything in the second photo was added outside of the lab. Transparency, motion blur, depth of field, and the “inverse normal” trick to make a glass sphere. The time is actually measured without Russian roulette (therefore I send two separate rays in case of refraction, and having 2 transparent spheres, it takes a lot of time...). The light source itself is a sphere, which is why the shadows (especially for the transparent sphere) are so grainy.

2.3 Lab 3



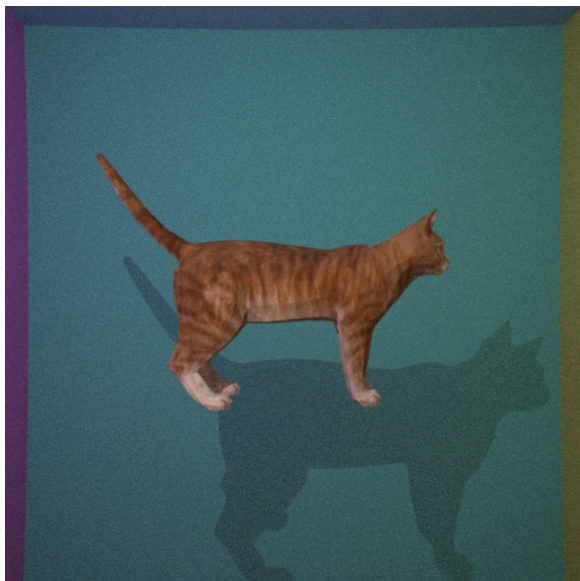
Execution metrics:

```
real 1m54.344s
user 37m27.170s
sys 0m0.888s
```

Figure 4: Result at the end of lab 3. Cat with 3954 triangles.

This lab adds triangle meshes. We can now import a .obj file, and we just had to implement the intersection function following some formulas. The running time is pretty bad, but currently we are traversing all 3954 triangles for every ray which hits the bounding box of our object.

2.4 Lab 4



Execution metrics:

```
real 0m3.971s
user 1m16.903s
sys 0m0.053s
```

Figure 5: BVH reduced running time considerably, and added textures.

Because Bounded Volume Hierarchy (BVH) takes us from linear time when a ray hits the bounding box to roughly-logarithmic by segmenting the cats in smaller bounding boxes, we observe a massive speed-up of about 28x.

2.5 Miscellaneous stuff

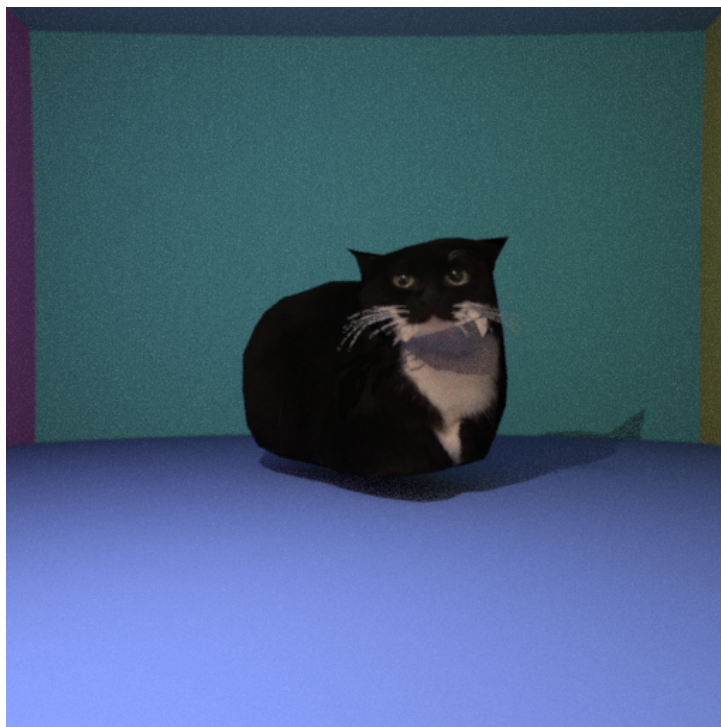


Figure 6: 144-frame animation of Dingus using ffmpeg, available [here](#).

2.6 Potential improvements

It is far from optimal (for example, each `std::default_random_engine` could be `thread_local`, which would remove the cache misses and incur a considerable speed-up of about 30% post-BVH, when this becomes a bottleneck). Moreover, it is one huge 1100-line file which is ugly. Also, for transparency, the current code does Russian roulette, which is faster than shooting two different rays but I acknowledge it is worse with how many samples we are working with.

Upon further testing, by improving the math inside the `TriangleMesh` intersection, I could improve the efficiency by 2x (before BVH). However, I will not further modify the code.